Fast Sync 구현 분석

개념적 배경

Fast Sync는 Ethereum 노드가 전체 블록체인을 빠르게 동기화하기 위해 개발된 메커니즘입니다. 전통적인 Full Sync가 모든 블록을 순차적으로 다운로드하고 실행하는 반면, Fast Sync는 헤더 체인을 먼저 다운로드한 후 특정 피벗 포인트에서 상태를 직접 다운로드하는 방식을 사용합니다.

설계 필요성

1. **동기화 시간 단축**: 전체 블록 실행 없이 최신 상태에 빠르게 도달
2. **네트워크 효율성**: 필요한 데이터만 선택적으로 다운로드
3. **리소스 최적화**: CPU 집약적인 트랜잭션 실행을 피함
4. **사용자 경험 개선**: 노드 시작 시간을 대폭 단축

다른 접근법과의 비교

- **Full Sync**: 모든 블록을 실행하여 완전한 검증 제공
- **Snap Sync**: 스냅샷 기반으로 더욱 빠른 동기화 제공
- **Light Sync**: 헤더만 다운로드하여 최소한의 검증 제공

구현 분석

핵심 데이터 구조

Downloader 구조체

```go
type Downloader struct {
mode atomic.Uint32 // 동기화 모드 (FullSync/SnapSync)
mux *event.TypeMux // 이벤트 멀티플렉서
queue *queue // 다운로드 스케줄러
peers *peerSet // 활성 피어 집합
stateDB ethdb.Database // 상태 동기화용 데이터베이스
// 통계 정보
syncStatsChainOrigin  uint64  // 동기화 시작 블록 번호
syncStatsChainHeight  uint64  // 알려진 최고 블록 번호
blockchain BlockChain // 블록체인 인터페이스
// 피벗 헤더 (Fast Sync용)
pivotHeader *types.Header
pivotLock sync.RWMutex
// 상태 동기화
SnapSyncer *snap.Syncer
stateSyncStart  chan *stateSync
// 스켈레톤 동기화 (Beacon 체인용)
skeleton *skeleton
}
```

동기화 큐 구조체

```go
type queue struct {
```

```
mode SyncMode // 동기화 모드
headerHead common.Hash // 마지막 큐잉된 헤더 해시
// 블록 바디 검색 작업
blockTaskPool  map[common.Hash]*types.Header
blockTaskQueue *prque.Prque[int64, *types.Header]
blockPendPool  map[string]*fetchRequest
// 영수증 검색 작업
receiptTaskPool  map[common.Hash]*types.Header
receiptTaskQueue *prque.Prque[int64, *types.Header]
receiptPendPool  map[string]*fetchRequest
resultCache *resultStore // 다운로드된 결과 캐시
}
```

주요 알고리즘

1. 헤더 체인 다운로드 알고리즘

Fast Sync는 다음 단계로 헤더 체인을 구축합니다:

1. **피어 선택**: 가장 높은 블록 번호를 가진 피어 식별
2. **헤더 요청**: 배치 단위로 헤더 요청 (기본 192개)
3. **헤더 검증**: 부모-자식 관계 및 PoW 검증
4. **체인 연결**: 연속적인 헤더 체인 구성

```
// fetchHeadersByHash는 해시로 헤더를 가져오는 블로킹 함수
func (d *Downloader) fetchHeadersByHash(p *peerConnection, hash common.Hash,
```

```go
amount int, skip int, reverse bool) ([]*types.Header, []common.Hash, error) {
// 응답 채널 생성 및 네트워크 요청 전송
resCh := make(chan *eth.Response)
req, err := p.peer.RequestHeadersByHash(hash, amount, skip, reverse, resCh)
if err != nil {
return nil, nil, err
}
defer req.Close()
// 타임아웃 설정
ttl := d.peers.rates.TargetTimeout()
timeoutTimer := time.NewTimer(ttl)
defer timeoutTimer.Stop()
select {
case <-d.cancelCh:
return nil, nil, errCanceled
case <-timeoutTimer.C:
return nil, nil, errTimeout
case res := <-resCh:
// 헤더 성공적으로 수신
return *res.Res.(*eth.BlockHeadersRequest), res.Meta.([]common.Hash), nil
}
}
```

2. 피벗 포인트 선택 전략

Fast Sync는 피벗 포인트를 선택하여 상태 동기화의 시작점을 결정합니다:

```
// 피벗 헤더 설정
func (d *Downloader) setPivotHeader(header *types.Header) {
d.pivotLock.Lock()
defer d.pivotLock.Unlock()
d.pivotHeader = header
}

// 피벗 헤더 조회
func (d *Downloader) getPivotHeader() *types.Header {
d.pivotLock.RLock()
defer d.pivotLock.RUnlock()
return d.pivotHeader
}
```

피벗 포인트 선택 기준:

- 충분히 확정된 블록 (일반적으로 64블록 이전)
- 네트워크 합의가 이루어진 블록
- 상태 크기가 적절한 블록

3. 상태 동기화 메커니즘

피벗 포인트가 결정되면 해당 블록의 상태를 직접 다운로드합니다:

```go
// syncState는 주어진 루트 해시로 상태 동기화를 시작
func (d *Downloader) syncState(root common.Hash) *stateSync {
s := newStateSync(d, root)
select {
case d.stateSyncStart <- s:
<-s.started // 시작 대기
case <-d.quitCh:
s.err = errCancelStateFetch
close(s.done)
}
return s
}


// stateSync 구조체
type stateSync struct {
d *Downloader
root common.Hash // 동기화할 상태 루트
started chan struct{} // 시작 신호
cancel chan struct{} // 취소 신호
done chan struct{} // 완료 신호
err error // 에러 정보
}
```

코드 구조 및 모듈 관계

```
eth/downloader/
├── downloader.go # 메인 다운로더 로직
├── queue.go # 다운로드 큐 관리
├── fetchers.go # 데이터 페처 구현
├── statesync.go # 상태 동기화 로직
├── skeleton.go # 스켈레톤 동기화 (Beacon)
├── peer.go # 피어 관리
└── events.go # 이벤트 정의
```

모듈 간 의존성:

- `downloader.go` → 모든 모듈의 중앙 조정자
- `queue.go` → 다운로드 작업 스케줄링
- `fetchers.go` → 네트워크 통신 담당
- `statesync.go` → 상태 데이터 동기화

실행 흐름

단계별 실행 과정

1단계: 동기화 초기화

```go
func (d *Downloader) synchronise(id  string, head common.Hash, td *big.Int, mode
// 동기화 모드 설정
```

```
d.mode.Store(uint32(mode))

// 피어 검증

p := d.peers.Peer(id)

if p == nil {

return errUnknownPeer

}

// 동기화 시작

return d.syncWithPeer(p, head, td)

}
```

2단계: 헤더 체인 구축

1. **최신 헤더 요청**: 피어로부터 최신 헤더 정보 수집
2. **헤더 다운로드**: 배치 단위로 헤더 다운로드
3. **헤더 검증**: PoW 및 체인 연결성 검증
4. **헤더 삽입**: 로컬 체인에 헤더 삽입

```
// 헤더 처리 파이프라인

func (d *Downloader) processHeaders(origin uint64, td *big.Int, ttl time.Durati

beaconMode bool) error {

for {

select {

case task := <-d.headerProcCh:

// 헤더 배치 처리
```

```
if err := d.processHeaderBatch(task.headers, task.hashes); err != nil {
return err
}
case <-d.cancelCh:
return errCanceled
}
}
}
```

3단계: 피벗 포인트 결정

```
// 피벗 포인트 계산
func (d *Downloader) calculatePivot(head *types.Header) *types.Header {
// 안전한 거리만큼 뒤로 이동 (일반적으로 64블록)
pivotNumber := head.Number.Uint64()
if pivotNumber > fsMinFullBlocks {
pivotNumber -= fsMinFullBlocks
}
// 피벗 헤더 조회
return d.blockchain.GetHeaderByNumber(pivotNumber)
}
```

4단계: 상태 동기화

1. **상태 루트 식별**: 피벗 블록의 상태 루트 추출

2. **상태 다운로드**: 트라이 노드 및 계정 데이터 다운로드

3. **상태 검증**: 다운로드된 상태의 무결성 검증

4. **상태 커밋**: 로컬 데이터베이스에 상태 저장

5단계: 블록 바디 및 영수증 다운로드

```
// 블록 바디 다운로드
func (d *Downloader) fetchBodies(from  uint64) error {
// 다운로드할 헤더 수집
headers := d.queue.PendingBodies()
// 배치 단위로 바디 요청
for  len(headers) > 0 {
batch := headers[:min(len(headers), MaxBlockFetch)]
if  err := d.fetchBlockBodies(batch);  err != nil {
return  err
}
headers = headers[len(batch):]
}
return  nil
}
```

상태 변화 추적

Fast Sync 과정에서의 주요 상태 변화:

1. **Idle → Synchronising**: 동기화 시작

2. **Header Sync**: 헤더 체인 구축 중
3. **Pivot Selection**: 피벗 포인트 결정
4. **State Sync**: 상태 데이터 동기화 중
5. **Body/Receipt Sync**: 블록 바디 및 영수증 동기화
6. **Completed**: 동기화 완료

에러 처리 메커니즘

주요 에러 타입

```
var (
errBusy = errors.New("busy")
errBadPeer = errors.New("action from bad peer ignored")
errTimeout = errors.New("timeout")
errInvalidChain = errors.New("retrieved hash chain is invalid")
errInvalidBody = errors.New("retrieved block body is invalid")
errInvalidReceipt = errors.New("retrieved receipt is invalid")
errCancelStateFetch = errors.New("state data download canceled")
errNoPivotHeader = errors.New("pivot header is not found")
)
```

에러 복구 전략

1. **피어 드롭**: 악의적이거나 불안정한 피어 제거
2. **재시도 로직**: 일시적 네트워크 오류에 대한 재시도
3. **피벗 재선택**: 피벗 포인트 문제 시 새로운 피벗 선택

4. **부분 롤백**: 손상된 데이터 감지 시 부분적 롤백

성능 최적화

적용된 최적화 기법

1. 병렬 다운로드

```
const (
MaxBlockFetch = 128  // 블록 페치 배치 크기
MaxHeaderFetch = 192  // 헤더 페치 배치 크기
MaxReceiptFetch = 256  // 영수증 페치 배치 크기
)
```

- 여러 피어로부터 동시에 데이터 다운로드
- 배치 크기 최적화로 네트워크 효율성 향상
- 피어별 성능에 따른 동적 할당

2. 메모리 관리

```
var (
blockCacheMaxItems = 8192  // 최대 캐시 블록 수
blockCacheInitialItems = 2048  // 초기 캐시 블록 수
blockCacheMemory = 256 * 1024 * 1024  // 최대 메모리 사용량 (256MB)
)
```

- 블록 캐시를 통한 메모리 사용량 제어

- 동적 캐시 크기 조정
- 메모리 압박 시 자동 정리

3. 네트워크 최적화

```
// 피어 성능 기반 타임아웃 계산
func (ps *peerSet) TargetTimeout() time.Duration {
// 피어 응답 시간 통계 기반 동적 타임아웃
return  ps.rates.TargetTimeout()
}
```

- 피어별 성능 모니터링
- 동적 타임아웃 조정
- 대역폭 기반 작업 할당

병목 지점 분석

1. 네트워크 병목

- **문제**: 느린 피어로 인한 전체 동기화 지연
- **해결**: 피어 성능 모니터링 및 동적 재할당

2. 디스크 I/O 병목

- **문제**: 대량의 상태 데이터 쓰기로 인한 지연
- **해결**: 배치 쓰기 및 압축 사용

3. CPU 병목

- **문제**: 헤더 검증 및 해시 계산

- **해결**: 병렬 검증 및 캐시 활용

튜닝 가능한 매개변수

네트워크 관련 매개변수

```
// 조정 가능한 상수들
const (
maxQueuedHeaders = 32 * 1024  // 큐잉 가능한 최대 헤더 수
maxHeadersProcess = 2048  // 한 번에 처리할 헤더 수
maxResultsProcess = 2048  // 한 번에 처리할 결과 수
fsHeaderSafetyNet = 2048  // 헤더 안전망 크기
fsMinFullBlocks = 64  // 최소 풀 블록 수
)
```

성능 튜닝 가이드

1. **높은 대역폭 환경**: `MaxHeaderFetch`, `MaxBlockFetch` 증가
2. **낮은 메모리 환경**: `blockCacheMaxItems` 감소
3. **느린 디스크**: 배치 크기 조정으로 I/O 최적화
4. **불안정한 네트워크**: 타임아웃 값 증가

실제 사용 예시

코드 예제

Fast Sync 시작

```
// Fast Sync 모드로 동기화 시작
func startFastSync(downloader *Downloader, peerID string, head common.Hash, td
// Fast Sync 모드 설정 (현재는 SnapSync가 기본)
mode := ethconfig.SnapSync
// 동기화 시작
err := downloader.synchronise(peerID, head, td, mode)
if err != nil {
return fmt.Errorf("fast sync failed: %v", err)
}
return nil
}
```

동기화 진행 상황 모니터링

```
// 동기화 진행 상황 확인
func monitorSyncProgress(downloader *Downloader) {
ticker := time.NewTicker(10 * time.Second)
defer ticker.Stop()
for {
select {
case <-ticker.C:
progress := downloader.Progress()
fmt.Printf("동기화 진행률: %.2f%%\n",
```

```
float64(progress.CurrentBlock-progress.StartingBlock)/
float64(progress.HighestBlock-progress.StartingBlock)*100)
fmt.Printf("현재 블록: %d, 최고 블록: %d\n",
progress.CurrentBlock, progress.HighestBlock)
if progress.CurrentBlock >= progress.HighestBlock {
fmt.Println("동기화 완료!")
return
}
}
}
}
```

설정 방법

geth 명령행 옵션

```
# Snap Sync 모드로 시작 (기본값)
geth --syncmode snap

# Full Sync 모드로 시작
geth --syncmode full

# 캐시 크기 조정
geth --cache 1024
```

```
# 최대 피어 수 설정
geth  --maxpeers  50
```

프로그래밍 방식 설정

```
// 이더리움 노드 설정
config := &ethconfig.Config{
SyncMode: ethconfig.SnapSync,
DatabaseCache: 1024,
TrieCleanCache: 256,
TrieDirtyCache: 512,
}

// 노드 생성
node, err := node.New(&node.Config{})
if  err != nil {
log.Fatal(err)
}

// 이더리움 서비스 등록
ethService, err := eth.New(node, config)
if  err != nil {
```

```
log.Fatal(err)

}
```

문제 해결 가이드

일반적인 문제와 해결 방법

1. **동기화가 멈춤**

- 원인: 피어 연결 문제 또는 네트워크 불안정
- 해결: 피어 재연결 또는 다른 부트노드 사용

2. **메모리 부족 오류**

- 원인: 캐시 크기가 너무 큼
- 해결: `--cache` 옵션으로 캐시 크기 감소

3. **디스크 공간 부족**

- 원인: 상태 데이터 크기 증가
- 해결: 충분한 디스크 공간 확보 또는 프루닝 활성화

4. **동기화 속도 저하**

- 원인: 느린 피어 또는 네트워크 병목
- 해결: 더 많은 피어 연결 또는 네트워크 최적화

로그 분석

```
// 동기화 관련 로그 모니터링

log.Info("Fast sync started", "mode", mode, "pivot", pivotHeader.Number)

log.Info("State sync progress", "accounts", progress.AccountSynced,

"storage", progress.StorageSynced)
```

```
log.Info("Block sync progress", "current", current, "highest", highest)
```

관련 시스템

의존성 관계

상위 의존성

- **eth/handler.go**: 동기화 요청 처리
- **eth/backend.go**: 이더리움 백엔드 서비스
- **core/blockchain.go**: 블록체인 상태 관리

하위 의존성

- **eth/protocols/eth**: 이더리움 프로토콜 구현
- **core/rawdb**: 데이터베이스 접근 계층
- **core/types**: 블록 및 트랜잭션 타입 정의

상호작용 패턴

P2P 네트워킹과의 상호작용

```
// 피어로부터 헤더 요청
func (p *peerConnection) RequestHeadersByHash(hash common.Hash, amount int,
skip int, reverse bool, sink chan *eth.Response) (*eth.Request, error) {
return p.peer.RequestHeadersByHash(hash, amount, skip, reverse, sink)
}
```

블록체인과의 상호작용

```
// 블록체인에 헤더 삽입
func (d *Downloader) insertHeaders(headers []*types.Header) error {
return  d.blockchain.InsertHeaderChain(headers)
}

// 현재 블록 상태 조회
func (d *Downloader) getCurrentBlock() *types.Header {
return  d.blockchain.CurrentBlock()
}
```

상태 관리와의 상호작용

```
// 상태 동기화 시작
func (d *Downloader) startStateSync(root common.Hash) {
stateSync := d.syncState(root)
go  stateSync.run()
}
```

영향 범위

직접적 영향

- **동기화 성능**: Fast Sync 효율성이 전체 노드 시작 시간 결정

- **네트워크 사용량**: 다운로드 패턴이 네트워크 부하에 영향
- **메모리 사용량**: 캐시 및 큐 크기가 메모리 사용량 결정

간접적 영향

- **사용자 경험**: 빠른 동기화로 노드 사용성 향상
- **네트워크 건강성**: 효율적인 동기화로 네트워크 부하 분산
- **개발자 생산성**: 빠른 테스트 환경 구축 가능

Fast Sync는 Ethereum 노드의 핵심 기능으로, 네트워크 참여 장벽을 낮추고 사용자 경험을 크게 개선하는 중요한 메커니즘입니다. 헤더 체인 다운로드, 피벗 포인트 선택, 상태 동기화의 3단계 과정을 통해 효율적이고 안전한 블록체인 동기화를 제공합니다.